akris61

CS 498 HW3

1. Row Key

Row key: City#Make#DOL Vehicle ID

Example: Seattle#TESLA#254622924

DOL Vehicle ID is unique, so no two rows have the same key.

It doesn't start with a timestamp or an increasing ID. So new writes don't all hit one node.

There are 935 cities, so writes get spread out. Seattle is big (15%) but it can still be split.

City queries are fast. They are just a prefix scan like Seattle#.

City + Make is also a prefix scan, like Seattle#TESLA#.

Make by itself is slow. It needs a full scan. A second table keyed by Make#City#ID would fix that.

2. Column Families

vehicle: VIN, Make, Model, Model Year, EV Type, Electric Range

location: County, City, State, Postal Code, Legislative District, Vehicle Location, GEOID

registration: DOL Vehicle ID, CAFV Eligibility, Electric Utility

Each family is stored separately. A query that only needs vehicle info skips the rest.

So less data gets read and sent between nodes.

3. Hashed vs Range Shard Key

Hashed (VIN):

+ Writes are spread evenly.

+ Looking up one VIN goes to one shard.

- Range queries have to go to every shard.

Range (Model Year):

+ Year range queries only hit the shards they need.

- About 72% of cars are 2022 or newer. New writes pile onto one shard.

- Only 23 years, so it can't be split much.

4. Performance

500 requests to each endpoint, one at a time.

On the VM (localhost):

insert-fast avg: 18.48 ms

insert-safe avg: 20.17 ms

From my laptop (public IP):

insert-fast avg: 99.11 ms

insert-safe avg: 105.87 ms

insert-safe is slower. It waits for a secondary to get the write.

On the VM the median went from 4.77 ms to 7.03 ms. A few slow requests pushed the averages up.

From my laptop, internet lag was bigger than the difference.

5. When to use the fast write

w=1 can lose a write if the primary crashes first.

With CAP, it picks availability and speed over consistency.

I'd use it for EV chargers sending status every few seconds.

Losing a few readings is fine. A new one comes right after.

For payments I'd use majority.